Axona API Reference

Reference for every public symbol exported from `@axona/protocol` v3.5.0.

Organized by what application developers reach for first (identity, peer lifecycle, pub/sub, direct messaging, introspection), then the transport/protocol surface, then low-level utilities. Every signature below is verified against the v3.5.0 kernel source.

Companion documents:

- Quick Start — 5-minute working roundtrip.
- Programmer Guide — mental model + worked example + pitfalls.
- Security changelog — what each kernel version protects.

Imports throughout assume:

```
import { /* … */ } from '@axona/protocol';
```

Sub-path imports (e.g. `@axona/protocol/contracts/Transport.js`, `@axona/protocol/persistence/file.js`) work for symbols that need them, but most apps only need the main barrel.

What changed since v2.x

The pub/sub and identity surfaces were redesigned in the v3 line. If you are migrating, the load-bearing differences are:

- **Two identity factories.** `createNodeIdentity({ lat, lng })` mints the *connection* key (the nodeId); `createAuthorIdentity()` mints a location-free *authorship* key (the Author ID). The old single `deriveIdentity()` is gone.
- **Topics are descriptors, not strings.** A topic is `{ region?, owner?, name, write? }`. `peer.pub` takes the descriptor; `peer.sub` / `pull` / `metrics` take the descriptor **or** a 66-hex topic ID. There is no `publisher`/`publishId` argument anywhere.
- **Signing is explicit per publish.** `peer.pub(topic, message, { signWith })` requires a signer — an author identity or the `ANONYMOUS` sentinel. There is no default author, the node key never signs publishes, and there is no `sign: false`.
- **The envelope carries the topic descriptor.** A signed publish binds the exact `{ region, owner, name, write }`, so a storing node recomputes the topic ID and enforces the write policy (`signer === owner` for owned topics).

Table of contents

Application surface (what you'll use first)

7. Envelopes + verification
8. Errors
9. Persistence

Transport + protocol surface

10. Contracts
11. Sim transport
12. Web transport
13. Handshake
14. Authenticated-identity handshake (axona/5)
15. Bridge directory
16. Low-level pub/sub builders

Low-level utilities

17. Ed25519 helpers
18. Hex / ID math
19. S2 geographic cells
20. Region names
21. Geo helpers
22. Proof-of-work scaffolding
23. Constants

Application surface

1. Types referenced throughout

`Identity` (node identity)

Returned by `createNodeIdentity`. The connection/transport keypair; its pubkey forms the nodeId.

```
{
  id:         string;       // 66-char lowercase hex (264-bit nodeId)
  pubkey:     Uint8Array;   // 32-byte Ed25519 public key
  pubkeyHex:  string;       // 64-char lowercase hex
  privateKey: CryptoKey;    // Web Crypto Ed25519 (browser + Node 20+)
  region:     { lat: number, lng: number };
  createdAt:  number;       // ms epoch
  pow:        string;       // transport-role PoW nonce ('' = inert at difficulty 0)
  sign(message: Uint8Array): Promise<Uint8Array>;
  verify(message: Uint8Array, signature: Uint8Array): Promise<boolean>;
}
```

The top byte of `id` is the S2 region cell for `(lat, lng)`; the bottom 256 bits are `SHA-256(pubkey)`. The id is self-authenticating — a peer can only claim an id it holds the private key for.

`AuthorIdentity` (authorship identity)

Returned by `createAuthorIdentity`. A keypair only — **no nodeId, no region** (authorship is not a place). Its public key is the Author ID.

```
{
  kind:       'author';
  authorId:   string;       // 64-char hex Author ID (== signerPubkey on the wire)
  pubkey:     Uint8Array;   // 32-byte Ed25519 public key
```

```
  pubkeyHex:  string;       // 64-char hex (same value as authorId)
  privateKey: CryptoKey;
  createdAt:  number;
  pow:        string;       // publish-role PoW nonce ('' = inert)
  sign(message: Uint8Array): Promise<Uint8Array>;
  verify(message: Uint8Array, signature: Uint8Array): Promise<boolean>;
}
```

You pass an `AuthorIdentity` (or `ANONYMOUS`) as `{ signWith }` to `peer.pub` / `kill` / `touch` / `unpub`.

`TopicDescriptor` (type)

```
{
  region?: string | number;   // region name ('useast'), '0x89', '137', or 137; omit → node region
  owner?:  string | null;     // 64-hex Author ID, or absent for an open topic
  name:    string;            // required, non-empty
  write?:  'open' | 'owner';  // defaults by owner presence (see §4.1)
}
```

`Envelope` (type)

What `peer.sub` delivers and `peer.pull` returns.

```
{
  msgId:        string;   // 64-char hex = sha256(canonical({ publisher, message }))
  seq:          number;   // per-publisher monotonic sequence (folded under the signature)
  ts:           number;   // ms epoch when published
  topic:        TopicDescriptor;   // the SIGNED descriptor { region, owner, name, write }
  message:      any;      // your JSON-serializable payload
  signature?:   string;   // 'ed25519:<128 hex>'; present iff signed
  signerPubkey?: string;  // 64-char hex Author ID; present iff signed
  signerPow?:   string;   // publish-role PoW nonce; present iff signed ('' at difficulty 0)
}
```

A **retraction** (see `peer.kill`) is delivered to your `sub` handler as a marker instead of a full envelope:

```
{ deleted: true, msgId: string, topic: string | null }
```

Branch on `env.deleted === true` to drop your local copy of `msgId`.

`Subscription` (type)

Opaque handle returned by `peer.sub()`.

```
sub.id          // string - unique per subscription
sub.topicId     // string - 66-char hex topic ID
sub.topicName   // string - your original topic name (or '#<id-prefix>' for an id-only sub)
sub.stop()      // Promise<void> - cancel; idempotent
```

`PeerId` / `TopicId` (types)

In the public API, peer IDs and topic IDs are **66-char lowercase hex strings**. The kernel holds them as `bigint` internally; conversion happens at the API boundary. Pass strings unless a signature explicitly says BigInt (`peer.lookup`).

---

2. Identity

Two factories. `createNodeIdentity` is the connection key (location + nodeId, used for the handshake and routing). `createAuthorIdentity` is the publish key (no location, used to sign messages). They are separate on purpose (key separation): a peer holds a node identity and signs each publish with an author identity supplied per call.

`createNodeIdentity({ lat, lng, extractable? }) → Promise<Identity>`

Generate a fresh node identity: a new Ed25519 keypair plus the 264-bit nodeId derived from the public key + region.

Param	Type	Notes
`lat`	`number`	required — latitude of the node's region
`lng`	`number`	required — longitude
`extractable`	`boolean`	default `true`. Pass `false` for an ephemeral browser identity so the signing key can't be exported (XSS can't exfiltrate it). Leave `true` if you will `dumpIdentity` it.

Returns an `Identity`. The top byte of `.id` is the S2 cell for `(lat, lng)`.

Throws `IdentityError`:

- `IDENTITY_INVALID_FORMAT` — `lat`/`lng` not numbers.
- `IDENTITY_KEYGEN_FAILED` — Web Crypto Ed25519 `generateKey` failed.

```
import { createNodeIdentity } from '@axona/protocol';

const node = await createNodeIdentity({ lat: 38.0, lng: -77.0 });
console.log(node.id);              // 'df…'  (66 hex chars)
console.log(node.id.slice(0, 2)); // region byte
```

`createAuthorIdentity({ persistAs?, store?, extractable? }) → Promise<AuthorIdentity>`

Mint (or load-or-create) the authorship key whose public key is the Author ID. Durability is the only real choice: ephemeral (unlinkable) or persisted (a recognizable author across sessions, able to retract).

Param	Type	Notes
`persistAs`	`string`	storage key; load-or-create then save. Omit for an ephemeral author.
`store`	`{ get, set }`	custom store; defaults to browser `localStorage` when `persistAs` is set.
`extractable`	`boolean`	default `true`; **forced `true`** when `persistAs` is set (a persisted key must be exportable).

Returns an `AuthorIdentity`.

Throws `IdentityError` (`IDENTITY_KEYGEN_FAILED`, `IDENTITY_LOAD_FAILED`, `IDENTITY_INVALID_FORMAT`).

```
import { createAuthorIdentity } from '@axona/protocol';

const ephemeral = await createAuthorIdentity();              // unlinkable, one session
const durable   = await createAuthorIdentity({ persistAs: 'me' }); // recognizable across reloads
console.log(durable.authorId);   // 64-hex Author ID - share this as `owner`
```

`dumpIdentity(identity) → Promise<IdentityEnvelope>` *(advanced)*

Serialize a **node** identity to a JSON-safe envelope (the `privateKey` is exported as base64 PKCS#8). Loses the in-memory `CryptoKey` handle; reconstruct with `loadIdentity`.

```
const env = await dumpIdentity(node);
// { id, pubkey, privkey, region: { lat, lng }, createdAt, pow }
localStorage.setItem('node-identity', JSON.stringify(env));
```

Throws `IdentityError(IDENTITY_LOAD_FAILED)` if the key can't be exported.

`loadIdentity(envelope) → Promise<Identity>` *(advanced)*

Inverse of `dumpIdentity`. Reconstructs the full node `Identity`, verifying that (a) the stored id matches the freshly-derived id and (b) the private key corresponds to the public key (a sign→verify probe).

```
const env = JSON.parse(localStorage.getItem('node-identity'));
const node = await loadIdentity(env);
```

Throws `IdentityError`:

- `IDENTITY_INVALID_FORMAT` — malformed envelope, id mismatch, or private/public key mismatch.
- `IDENTITY_LOAD_FAILED` — PKCS#8 import failed.

  Author-identity persistence is handled inside `createAuthorIdentity({ persistAs })` — you don't dump/load author identities by hand. `dumpIdentity`/`loadIdentity` are for node identities (or when you manage your own storage layer).

`computeNodeId(pubkeyBytes, lat, lng) → Promise<string>` *(advanced)*

`computeNodeIdBigInt(pubkeyBytes, lat, lng) → Promise<bigint>` *(advanced)*

Derive (or verify) a 264-bit nodeId from a raw public key + region. `computeNodeId` returns the 66-hex form; `computeNodeIdBigInt` returns the BigInt. Used to validate a claimed nodeId against a pubkey + region without minting a fresh identity.

```
const id = await computeNodeId(node.pubkey, 38.0, -77.0);
console.log(id === node.id);   // true
```

---

3. AxonaPeer construction + lifecycle

`AxonaPeer` is the per-node DHT contract implementation — one instance per running node.

`new AxonaPeer({ nodeIdentity, domain, transport, ... })`

Param	Type	Notes
`node`	`NeuronNode`	**required** — the per-node state object (`{ id, alive, transport, synaptome }`).

Param	Type	Notes
`nodeIdentity`	`Identity`	the connection key (from `createNodeIdentity`). Used for the handshake, routing, and signing the node's own actions. It **never** signs a publish.
`transport`	`Transport`	the transport instance (web/sim/node). The peer uses it for `send`/`notify`/`lookup`.
`domain`	`AxonaDomain`	shared routing parameters. **`domain` or `engine` is required.**
`engine`	object	legacy simulator engine; pass instead of `domain` only in sim/tests.
`axonaManager`	`AxonaManager`	pre-built pub/sub manager; if omitted, resolved on first `pub`/`sub`.
`persist`	`PersistenceAdapter`	enables auto-checkpointing of identity, subscriptions, synaptome, and hosting state.
`maxPublishBytes`	`number`	per-publish cap; clamped to the WebRTC-interop floor (16 KiB) and never above the absolute ingress cap. Override only for known-homogeneous fleets.

The production path is `{ nodeIdentity, domain, transport }`. The `{ engine }` / `{ axonaManager }` forms are for the simulator and tests.

```
import { AxonaPeer, AxonaDomain, NeuronNode, createNodeIdentity } from '@axona/protocol';

const node = await createNodeIdentity({ lat: 38, lng: -77 });
const peer = new AxonaPeer({
  nodeIdentity: node,
  domain:       new AxonaDomain({ k: 20 }),
  node:         new NeuronNode({ id: BigInt('0x' + node.id), lat: 38, lng: -77 }),
  transport,
});
```

Throws plain `Error` if `node` is missing, or if neither `engine` nor `domain` is supplied.

`peer.start() → Promise<void>`

Bring the peer up — installs wire-level routing handlers, the delivery hook, the peer-bound/peer-died eviction handlers, and (if `persist` is wired) hydrates persisted state. Idempotent.

`peer.stop() → Promise<void>`

Tear down: release event listeners and the peer-lifecycle hooks. Idempotent. (For a graceful network exit, prefer `peer.leave`.)

`peer.join(sponsor?) → Promise<void>`

Bootstrap into the mesh. Calls `start()` first if needed, brings up the transport, and — if a `sponsor` is given — opens a channel to it and seeds the synaptome.

Param	Type	Notes
`sponsor`	`string`	66-char hex nodeId of a known peer. Omit to start standalone (inbound connections will populate the synaptome).

```
await peer.join();                  // standalone; wait for inbound peers
await peer.join(bridgeNodeIdHex);   // open a channel to a sponsor and seed
```

Throws `TransportError`:

- `TRANSPORT_NOT_STARTED` — `transport.start` failed, or `sponsor` is not 66-char hex.
- `TRANSPORT_PEER_UNREACHABLE` — the sponsor isn't reachable.

`peer.leave({ drain?, notify?, timeoutMs? }) → Promise<void>`

Graceful shutdown.

Param	Type	Default	Notes
`notify`	`boolean`	`true`	send a `peer-leaving` notification to every synaptome peer so they drop us proactively (sub-second handoff instead of heartbeat timeout).
`drain`	`boolean`	`true`	pause briefly for in-flight publishes/pulls to settle before closing.
`timeoutMs`	`number`	5000	bound on the drain wait.

Closes the transport last, force-flushes persistence, and stops listeners.

```
await peer.leave({ drain: true, notify: true, timeoutMs: 3000 });
```

`peer.getNodeId() → string | bigint`

Returns whatever was passed as `node.id` at construction time (BigInt if you followed the convention).

`peer.snapshot() → Promise<object>`

Serialize this peer's state to a JSON-safe envelope: `{ formatVersion: '1.0', snapshotAt, wireVersion, identity, synaptome, subscriptions }`. Store it however you like (encrypted, synced, custom DB).

```
const state = await peer.snapshot();
localStorage.setItem('peer-state', JSON.stringify(state));
```

`AxonaPeer.fromSnapshot(state, opts) → Promise<AxonaPeer>` *(static)*

Reconstruct a peer from a `snapshot()` envelope. The returned peer is constructed with identity / synaptome pre-loaded, but the transport is **not** started — call `peer.join(sponsor?)` to bring it up. Subscription **handlers are not restored** (functions don't serialize); the restored list is exposed at `peer.pendingSubscriptions` so you can re-register them with `peer.sub`.

```
const state = JSON.parse(localStorage.getItem('peer-state'));
const peer  = await AxonaPeer.fromSnapshot(state, { domain, node, transport });
for (const s of peer.pendingSubscriptions) {
  await peer.sub(s.topic, onMsg, { since: s.since });
}
await peer.join();
```

Throws `TypeError` / `RangeError` if `state` isn't a `1.0`-format snapshot object.

---

4. Pub/sub

4.1 Topic addressing

(Folded in from the former standalone "Topic IDs" note.)

A topic is not a bare string — it's a small **descriptor**, and the topic ID is a hash of that descriptor with a one-byte region prefix:

```
topicId = regionByte || SHA-256(canonical({ owner, name, write }))   // 33 bytes = 66 hex chars
```

Field	Meaning
`region`	a real geographic cell; becomes the leading byte of the ID
`owner`	an **Author ID** (a publish key), or absent
`name`	the human label — `"lobby"`, `"profile"`
`write`	`"open"` or `"owner"` — defaults by whether `owner` is set

Because the ID is a pure function of those fields, anyone who knows the fields computes the identical ID — no registry, no coordination.

The `region` field accepts a name, hex string, decimal string, or number — all normalize to the same region byte:

```
{ region: 'useast', name: 'lobby' }   // name
{ region: '0x89',   name: 'lobby' }   // hex string   --\  same topic ID
{ region: 137,      name: 'lobby' }   // number       --/  (leading byte 0x89)
```

Omit `region` and it defaults to the publisher's own node region (top byte of its node ID). Unknown regions throw `RangeError`.

`write` defaults by whether there's an `owner`:

Descriptor	Resolved `write`	Meaning
`{ region, name }` (no owner)	`open`	open lobby — anyone publishes
`{ region, owner, name }` (no write)	`owner`	owned — only the owner publishes
`{ region, owner, name, write:'owner' }`	`owner`	same ID as the row above
`{ region, owner, name, write:'open' }`	`open`	owner-namespaced, anyone publishes (inbox/wall)
`{ region, name, write:'owner' }` (no owner)	`open`	no owner -> `write` ignored

Two rules: **no `owner` -> the topic is open and `write` is ignored**; **an `owner` -> `write` defaults to `'owner'`** (the safe default — forgetting `write` can never silently make an owned feed world-writable).

Share the ID for reading; share the descriptor for writing. `sub`, `pull`, and `metrics` accept the 66-hex ID or a descriptor. `pub` (and owner ops `kill`/`unpub`) require the descriptor — a bare ID is rejected, because a hash can't reveal its `owner`, so the ID alone can't prove write authorization.

`deriveTopicId(descriptor, selfRegion?)` → `Promise<string>` Compute the 66-hex topic ID for a descriptor — the same function the peer uses internally. This is what you hand to someone so they can subscribe.

Param	Type	Notes
`descriptor`	`TopicDescriptor`	`{ region?, owner?, name, write? }`.
`selfRegion`	`string | number`	fallback region when `descriptor.region` is omitted (the publisher's node region).

```
import { deriveTopicId } from '@axona/protocol';

const lobbyId = await deriveTopicId({ region: 'useast', name: 'lobby' });
const feedId  = await deriveTopicId({ region: 'useast', owner: me.authorId, name: 'profile' });
// → "89…"  (66 hex chars; leading "89" is the region byte) - share out-of-band
```

Throws `TypeError` (empty `name`), `RangeError` (unknown region, bad `owner` format, or no region and no `selfRegion`).

`deriveTopicIdBig(descriptor, selfRegion?)` → `Promise<bigint>` BigInt variant of `deriveTopicId` (the form kernel internals hold). Same parameters and errors.

`resolveTopic(descriptor, selfRegion?)` → `Promise<object>` *(advanced)* The full resolver behind `deriveTopicId`. Returns the normalized descriptor plus the id: `{ region (code), owner, name, write, topicId }`. Use it when you need the resolved `write`/`region`, not just the id.

```
const r = await resolveTopic({ region: 'useast', owner: me.authorId, name: 'profile' });
// { region: 137, owner: '<64-hex>', name: 'profile', write: 'owner', topicId: '89…' }
```

`canonical(value)` → `string` *(advanced)* Stable, total, JSON-valid canonical encoding (object keys sorted at every level). Two semantically-identical values canonicalize to the same bytes across runs and implementations — the basis of content-addressed `msgId`s and signature stability.

`sha256Hex(input)` → `Promise<string>` *(advanced)* `SHA-256` of a string or `Uint8Array`, returned as lowercase hex.

4.2 publish / subscribe

`peer.pub(topic, message, { signWith })` → `Promise<string>` Publish a message to a topic. **Returns** the 64-char hex `msgId`.

Arg	Type	Notes
`topic`	`TopicDescriptor`	**must be a descriptor** — a bare id is rejected (the storing node needs the descriptor to verify the write policy).
`message`	`any`	JSON-serializable payload.
`opts.signWith`	`AuthorIdentity | ANONYMOUS`	**required** — name a signer, or pass the `ANONYMOUS` sentinel to publish unsigned. There is no default author.

```
import { createAuthorIdentity, ANONYMOUS } from '@axona/protocol';
const me = await createAuthorIdentity();

const id = await peer.pub({ region: 'useast', name: 'lobby' }, { text: 'hi' }, { signWith: me });
await peer.pub({ region: 'useast', name: 'lobby' }, 'anon msg', { signWith: ANONYMOUS });
```

For an **owned** topic, `signWith` must be the owner key — publishing with any other key throws before it leaves the peer.

Maximum size. The cap is on the *serialized envelope* and defaults to the WebRTC-interoperable reliable-delivery floor (16 KiB) so any peer on any path can receive it. Chunk larger payloads (`@axona/protocol/std/chunk`) or publish a content-reference and transfer bytes out-of-band.

Throws `PublishError`:

- `PUBLISH_INVALID_TOPIC` — a bare id was passed, or the topic isn't a `{ name, … }` object.
- `TOPIC_REGION_REQUIRED` — no region named and the peer has no node region to default to.
- `PUBLISH_NO_PUBLISH_IDENTITY` — no `signWith` given.
- `WRITE_POLICY_VIOLATION` — owned topic, signer is not the owner.
- `PUBLISH_SIGN_FAILED` — `signWith` lacks `privateKey`/`pubkeyHex`, or signing failed.
- `PUBLISH_INVALID_MESSAGE` — message isn't JSON-serializable.
- `PUBLISH_PAYLOAD_TOO_LARGE` — enveloped message exceeds the cap.

`peer.sub(topic, handler, { since? }) → Promise<Subscription>` Subscribe. `handler` is invoked with the full `Envelope` for each delivery.

Arg	Type	Notes
`topic`	`TopicDescriptor | string`	a descriptor **or** a 66-hex topic ID (the shareable read handle).
`handler`	`(envelope) => void`	called per delivery; also receives retraction markers `{ deleted: true, msgId, topic }`.
`opts.since`	`'all' | 'latest' | number`	replay control (below).

`since` modes:

`since`	What you get
omitted / `undefined`	live tail only — future messages.
`'latest'`	the most recent cached message, then live tail.
`'all'`	everything in the replay cache, then live tail.
`<number>`	messages newer than the timestamp (ms epoch), then live tail.

Returns a `Subscription`; call `.stop()` to cancel.

```
const sub = await peer.sub({ region: 'useast', name: 'lobby' }, (env) => {
  if (env.deleted) { dropFromUI(env.msgId); return; }
  console.log(env.signerPubkey, env.message);
}, { since: 'all' });

// or subscribe by shared id:
const sub2 = await peer.sub(lobbyId, onMsg, { since: 'latest' });

await sub.stop();
```

Throws `SubscribeError(SUBSCRIBE_HANDLER_MISSING)` (handler not a function); `PublishError(PUBLISH_INVALID_TOPIC` (a string that isn't a valid 66-hex id).

`peer.unsub(topic) → Promise<{ ok, removed }>` Unsubscribe by topic — the counterpart to `sub`. Stops **every** local subscription this peer holds for the topic (no need to keep the handle) and, once the last one goes, sends the network unsubscribe so the topic's roots drop this peer. Self-only by construction. Idempotent — returns `{ ok: true, removed: 0 }` for a topic you're not on.

Takes a **descriptor** (it derives the topicId the same way `sub`'s descriptor form does).

```
const { removed } = await peer.unsub({ region: 'useast', name: 'lobby' });
```

4.3 pull / metrics

`peer.pull(msgId, { topic, timeoutMs? }) → Promise<Envelope | null>` Fetch one message by content hash from the topic's K-closest replay cache. Returns `null` on a cache miss (including a message that aged out of the hold window) — that's expected, not an error.

Arg	Type	Notes
`msgId`	`string | null`	64-char hex, **or `null`/omitted for the topic's most-recent message**.
`opts.topic`	`TopicDescriptor | string`	descriptor **or** 66-hex id.
`opts.timeoutMs`	`number`	default `1000`.

A successful pull slides the message's hold time forward (bounded by the 48 h ceiling).

```
const env    = await peer.pull(msgId, { topic: feedId });
const latest = await peer.pull(null, { topic: feedId });   // most recent on the topic
```

Throws `PullError`:

- `PULL_INVALID_MSGID` — a non-null `msgId` that isn't 64-char hex.
- `PULL_AXONS_UNREACHABLE` — the manager can't service the request.

`peer.metrics(topic, { timeoutMs? }) → Promise<metricsObj>` ***(owner-only)*** Aggregate counters for a topic across the K-closest root set, **on demand**. As of kernel v3.5.0 this is an **owner-only** reader: the scatter-gather answers only the owner of an **owned** (`write:'owner'`) topic. **Open / public topics are refused** (they return an empty/zero result) — read their live state by subscribing to `metricTopic(T)` (above) instead. Use `metrics()` for an owner's private, immediate read of their own topic; it is not the path for public counts.

Arg	Type	Notes
`topic`	`TopicDescriptor | string`	descriptor **or** 66-hex id.
`opts.timeoutMs`	`number`	default `500`.

Returns:

```
{
  publishes:     number;   // distinct post hashes seen (cumulative; only rises)
  current_count: number;   // live (non-expired, non-killed) messages retained right now
  subscribers:   number;   // max direct-child count at any single responding relay
  deliveries:    number;   // total fan-out deliveries
  pulls:         number;    // total pull() hits
  reshares:      number;   // total reshare bumps
  relayCount:    number;   // distinct relays that replied (coverage sanity-check)
}
```

`current_count` falls as messages are killed or age out; `publishes` only rises. `subscribers` is exact for an unsplit topic, a lower bound once the tree splits. Use for UX, not billing.

```
// owner reading their OWN owned topic's live state, on demand:
const m = await peer.metrics({ region: 'useast', owner: me.authorId, name: 'inbox', write: 'owner' });
console.log(`${m.subscribers} subscribers, ${m.current_count} live messages`);
```

> **Owner-only (v3.5.0).** Open/public topics no longer answer this probe — a non-owner cannot trigger a K-root fan-out for any topic. For an open topic's public count, subscribe to `metricTopic(T)`. `metrics()` is the owner's private, immediate reader and the operator/debug escape hatch.

`metricTopic(dataTopicId)` → `TopicDescriptor` ***(subscribe to metrics — the scalable path)*** `peer.metrics()` above is a **scatter-gather**: one fan-out to the K roots per call. Fine for an occasional probe; **do not call it on a per-user timer** — the cost grows with both the audience and the poll rate. Instead **subscribe to a topic's metrics.** A topic's primary root publishes a signed metric snapshot to a *derived* topic on a ~5-min cadence; compute that topic with `metricTopic()` and `sub()` it. You get the latest snapshot via replay-on-subscribe plus every update — one subscription instead of a fan-out per poll — and, because snapshots are ordinary messages that age out at the 48 h hold ceiling, a **rolling ~48 h history for free** to plot trends.

```
import { deriveTopicId, metricTopic } from '@axona/protocol';

const id = await deriveTopicId({ region: 'useast', name: 'lobby' });
await peer.sub(metricTopic(id), (env) => {
  const m = JSON.parse(env.message);
  // { topic, ts, by, current_count, subscribers, bytes }
  render(m.subscribers, m.current_count);
}, { since: 'all' });   // since:'all' → latest snapshot + the rolling history
```

name	returns	notes
`metricTopic(dataTopicId)`	`TopicDescriptor`	open topic `{ region, name: 'axona:metric:' + id }`. Pass the **resolved 66-hex** id (`await deriveTopicId(desc)`), not the descriptor. Region byte is inherited from the data topic.

name	returns	notes
`isMetricTopic(descriptor)`	`boolean`	true if a topic is itself a metric topic (the recursion guard a relay uses).
`isMetricTopicName(name)` / `dataTopicIdOf(descriptor)`	`boolean` / `string|null`	name-prefix test; inverse (metric topic → data id).
`METRIC_NAMESPACE`	`string`	the frozen reserved prefix `'axona:metric:'`.

Trust is **advisory**: the metric topic is open (anyone may publish to it), so treat a snapshot as a hint — if you need to, pin trust to a known relay's `env.signerPubkey`. The protocol does not prove a snapshot is authoritative. Only **open** topics get a metric topic; owned-topic counts stay owner-only (read them with `peer.metrics()` as the owner). `peer.metrics()` remains the on-demand/operator escape hatch.

4.4 owner + creator ops: kill / touch / unpub

These take a **descriptor** (not a bare id) and a signer via `{ signWith }`.

`peer.kill(topic, msgId, { signWith }) → Promise<{ ok }>` Retract a message you published — "unsend". Authorized by **authorship**: the roots accept the kill only if its signer matches the signer of the cached message, so you can only kill messages **you** signed. The roots drop it from their replay cache, tombstone the `msgId` so a lagging replica can't resurrect it, and forward a delete marker to subscribers (`{ deleted: true, msgId, topic }`).

Arg	Type	Notes
`topic`	`TopicDescriptor`	the topic the message was published to.
`msgId`	`string`	**required** 64-char hex (the value `pub` returned).
`opts.signWith`	`AuthorIdentity`	the **same author key** that published the message.

```
const id = await peer.pub({ region: 'useast', name: 'lobby' }, { text: 'oops' }, { signWith: me });
await peer.kill({ region: 'useast', name: 'lobby' }, id, { signWith: me });
```

> Best-effort redaction, not a cryptographic un-send: a subscriber who already has the plaintext can keep it; an anonymous message has no provable creator and **cannot** be killed.

Throws `KillError`: `KILL_INVALID_MSGID` (bad msgId), `KILL_SIGN_FAILED` (no/invalid `signWith`, or signing failed). A network that can't authorize the kill is **not** an error — it just won't take effect.

`peer.touch(topic, msgId, { signWith }) → Promise<{ ok }>` Keep-alive. A signed touch routed to the roots resets the message's hold-time expiry to `now + hold` (bounded by the 48 h ceiling), moves it to the head of the queue, and makes it the last entry evicted. Use it to keep a still-relevant message alive past its default hold without re-publishing.

Authority is **by topic**, not by message authorship: on an **open** topic anyone may touch; on an **owned** topic only the owner may (verified at the root).

Arg	Type	Notes
`topic`	`TopicDescriptor`	the topic.
`msgId`	`string`	**required** 64-char hex.

Arg	Type	Notes
`opts.signWith`	`AuthorIdentity`	signs for freshness; on an owned topic must be the owner.

```
await peer.touch({ region: 'useast', name: 'status', owner: me.authorId }, id, { signWith: me });
```

Throws `TouchError`: `TOUCH_INVALID_MSGID`, `TOUCH_SIGN_FAILED`.

`peer.unpub(topic, { signWith, destroy? }) → Promise<{ ok }>` Remove an **owned** topic's message queue — owner-only. The roots verify ownership self-authenticatingly (`signer.pubkey === owner`).

Arg	Type	Notes
`topic`	`TopicDescriptor`	must be an owned topic (`{ owner, write: 'owner' }`).
`opts.signWith`	`AuthorIdentity`	must be the owner key.
`opts.destroy`	`boolean`	`false` (default) drops messages, keeps config/ACL so the owner can keep publishing; `true` is total removal (messages + config/ACL + hosting role state).

```
const feed = { region: 'useast', owner: me.authorId, name: 'profile' };
await peer.unpub(feed, { signWith: me });                  // clear the queue
await peer.unpub(feed, { signWith: me, destroy: true });   // remove entirely
```

Ownerless (open) topics have no owner key and **cannot** be unpubbed.

Throws `UnpubError`: `UNPUB_PUBLIC_TOPIC` (open topic), `UNPUB_SIGN_FAILED` (no/invalid `signWith`, or signer isn't the owner).

4.5 hosting: host / unhost

`peer.host(topic?, opts?) → Promise<{ ok, scope, topicId? }>` Store and serve a topic for other peers **without subscribing** — the relay / infrastructure primitive. The node becomes a willing root/replica (publishes land on it, subscribers pull replays from it) but registers **no handler** and delivers nothing to a local app.

- `peer.host()` — host this node's **own keyspace neighborhood**: get recruited as a root for whatever topics land near this node's id ("host whatever lands near me"). The zero-config relay mode; returns `{ ok: true, scope: 'keyspace' }`.
- `peer.host(topic)` — host one specific topic (descriptor). Returns `{ ok: true, scope: 'topic', topicId }`.

Wire-compatible with every kernel (reuses `subscribe-k`), respects the proximity gate, and is idempotent.

```
await peer.host();                                          // headless relay
await peer.host({ region: 'useast', name: 'pow-results' }); // host one topic
```

`peer.unhost(topic?, opts?) → Promise<{ ok, scope }>` Counterpart to `host`. `unhost()` turns off keyspace hosting; `unhost(topic)` drops one hosted topic. Does **not** touch your subscriptions. Idempotent.

`peer.rootedTopics() → Array<{ topicId, descriptor, current_count, subscribers, bytes }>` ***(infra)*** Synchronous, local-only introspection of the topics this node currently **roots** — each with its signed topic descriptor (recovered from a cached envelope, or `null` for an empty/cold role) and a locally-computed snapshot. No network (unlike `metrics()`). This is the read side that powers the relay metric-publish loop

(§4.3): walk it, skip `isMetricTopic(d)` and non-open descriptors, and `pub(metricTopic(topicId), …)` for the rest. Returns `[]` on a routing-only peer with no `AxonaManager`.

4.6 topic limits

Every topic's replay queue is bounded:

- **Max messages** — default 100, max 256. When full, the lowest-ordered message (by signed `seq`) is evicted deterministically. A max of 1 is a *retained/latest-value* slot; pair it with `peer.pull(null, …)`.
- **Hold time** — default 24 h, hard ceiling 48 h. A message past its hold is swept (stops being delivered or pulled). A `pull` or `touch` slides the hold forward, bounded by the ceiling.
- **Per-publisher quota (open topics only)** — one publisher occupies at most ¼ of the queue, so a flooder can't evict everyone else. Owner-gated topics aren't quota'd.

---

5. Direct messaging

1:1 messages between specific peers, bypassing pub/sub. `targetId` is the 66-char hex nodeId; the peer must already be in the synaptome.

`peer.send(targetId, message) → Promise<reply>`

Request/response. Resolves to whatever the recipient's `onMessage` handler returns.

```
const reply = await alice.send(bobIdHex, { kind: 'ping', body: 'hi' });
```

Throws `TypeError` if `targetId` isn't 66-char hex.

`peer.notify(targetId, message) → Promise<void>`

Fire-and-forget. Resolves once enqueued, not when delivered.

```
await alice.notify(bobIdHex, { kind: 'typing', user: 'alice' });
```

Throws `TypeError` if `targetId` isn't 66-char hex.

`peer.onMessage(handler) → void`

Register the single inbound direct-message handler. Calling it again **replaces** the previous handler. Signature `(senderId, message) => reply | void`: a returned value resolves the sender's `send()`; for `notify()` callers the return is discarded.

```
peer.onMessage(async (senderIdHex, message) => {
  switch (message?.kind) {
    case 'ping':   return { reply: 'pong' };   // resolves send()
    case 'typing': console.log(senderIdHex, 'typing'); return;  // notify discards
  }
});
```

`senderId` is the transport-authenticated sender, surfaced as 66-hex.

---

6. Mesh introspection + events

`peer.peers() → string[]`

Current synaptome membership as 66-char hex strings.

`peer.onPeerJoin(handler) → () => void`

Fires when a peer is admitted to the synaptome. Handler receives `(peerIdHex, event)`. Returns an unsubscribe function.

```
const off = peer.onPeerJoin((id, ev) => console.log('admitted', id, ev?.addedBy));
```

`peer.onPeerLeave(handler) → () => void`

Symmetric — fires on eviction (TTL, churn, explicit unbind). Same signature.

`peer.lookup(targetKey) → Promise<lookupResult>`

Iterative XOR-routed walk to find `targetKey` (typically a peer ID, but any 264-bit key works). `targetKey` is a BigInt.

```
const r = await peer.lookup(BigInt('0x' + targetHex));
// { found: boolean, hops: number, time: number /* ms */, path: bigint[] }
```

Returns `null` if the node isn't alive.

`peer.health() → healthObj`

A cheap synchronous diagnostic snapshot — safe to poll on a UI tick:

```
peer.health();
// {
//   nodeId, synaptomeSize, peers: [...], subscriptions,
//   axonRoles: [{ topic, isRoot, children, cacheSize }],
//   hosting:   { keyspace, topics } | null,
//   wireVersion, started,
//   transport: { boundCount, meshChannels, meshOpen, meshBound, bridgeState } | null,
//   meshDegraded: boolean,
// }
```

`meshDegraded === true` (open channels materially exceed authenticated binds) is the routing-truth signal — a single tick can be a mid-handshake transient; a value that stays `true` across polls is the real warning. `transport` is `null` on sim/node transports, populated on web.

`peer.onLog(level, handler) → () => void`

Subscribe to structured log events. **The level comes first** (`'debug' | 'info' | 'warn' | 'error'`); handler is `(msg, context?) => void`. Returns an unsubscribe function.

```
const off = peer.onLog('warn', (msg, ctx) => console.warn(msg, ctx));
```

Throws `TypeError` on an invalid level or non-function handler.

`peer.onError(handler) → () => void`

Fires on background `AxonaError` emissions (transport failures during heartbeat, persistence warnings) the kernel surfaces asynchronously rather than throwing. Returns an unsubscribe function.

`peer.onUpgradeRequired(handler) → () => void`

Fires on a wire-version handshake mismatch, with the `UpgradeRequiredError` carrying `{ reason, serverVersion, clientVersion, downloadUrl }`.

```
peer.onUpgradeRequired((err) => showUpgradeBanner(err.context.downloadUrl));
```

---

7. Envelopes + verification

The kernel builds and verifies envelopes for you inside `peer.pub` / `peer.sub`. These low-level helpers are for verifying what you consume, signing a DM payload by hand, or testing.

`buildEnvelope({ topic, message, ts?, seq?, identity, sign? }) → Promise<Envelope>`

Param	Type	Notes
`topic`	`TopicDescriptor`	the descriptor — signed, so the signature binds the exact write policy.
`message`	`any`	JSON-serializable.
`ts`	`number`	ms timestamp; defaults to `Date.now()`.
`seq`	`number`	per-publisher monotonic sequence (default `0`; `peer.pub` supplies a real value).
`identity`	`AuthorIdentity`	required when `sign: true`.
`sign`	`boolean`	default `true`.

Returns an `Envelope`. The signature covers a domain-tagged core (`axona:pubsub-envelope:v2`).

Throws `TypeError` if `identity` lacks `privateKey`/`pubkeyHex` when `sign: true`.

`verifyEnvelope(envelope) → Promise<{ ok, reason?, signed }>`

Verify the signature against the domain-tagged `(seq, ts, topic, message)` core and that `msgId` matches. **Returns a result object, not a boolean** — check `.ok`.

```
const res = await verifyEnvelope(env);
if (!res.ok) console.warn('forged/invalid', env.msgId, res.reason);
// res.signed === false for a valid unsigned envelope (res.ok === true)
```

> A common mistake: `if (!(await verifyEnvelope(env)))` is always false — the return value is a truthy object. Always test `.ok`.

`reason` (when `!ok`) is one of `not_an_object`, `missing_msgId`, `missing_ts`, `missing_topic`, `missing_message`, `missing_seq`, `missing_signerPubkey`, `unknown_signature_scheme`, `wrong_key_or_signature_length`, `bad_signature`, `bad_msgid`. `verifyEnvelope` checks signature + msgId but **not** freshness (the legitimate replay path serves cached history).

`computeMsgId({ publisher?, message }) → Promise<string>`

Stand-alone msgId computation matching `buildEnvelope`: `sha256(canonical({ publisher, message }))`, where `publisher` is the signer's pubkey (or `null` for anonymous). Time and seq are deliberately not folded in.

```
const msgId = await computeMsgId({ publisher: env.signerPubkey, message: env.message });
console.log(msgId === env.msgId);   // true
```

`checkFreshness(envelope, { now?, maxSkewMs? }) → { ok, reason? }`

Test whether an envelope's signed `ts` is within the freshness window (`MAX_PUBLISH_SKEW_MS`, 5 min by default). Live-publish ingress enforces this in the kernel; call it yourself only if you need the check on a path the kernel doesn't gate.

Envelope constants

- `ENVELOPE_DOMAIN` — `'axona:pubsub-envelope:v2'` (signature domain tag).
- `MAX_PUBLISH_SKEW_MS` — `300000` (the freshness window).

---

8. Errors

All thrown errors inherit from `AxonaError`:

```
class AxonaError extends Error {
  code:     string;   // stable UPPER_SNAKE identifier - switch on this, not message
  cause?:   Error;
  context?: object;   // machine-readable details
}
```

Subclasses and their codes:

Class	Codes
`IdentityError`	`IDENTITY_KEYGEN_FAILED`, `IDENTITY_LOAD_FAILED`, `IDENTITY_INVALID_FORMAT`
`TransportError`	`TRANSPORT_NOT_STARTED`, `TRANSPORT_PEER_UNREACHABLE`, `TRANSPORT_TIMEOUT`, `TRANSPORT_CHANNEL_CLOSED`, `TRANSPORT_HELLO_FAILED`
`PublishError`	`PUBLISH_INVALID_TOPIC`, `PUBLISH_SIGN_FAILED`, `PUBLISH_NO_PUBLISH_IDENTITY`, `TOPIC_REGION_REQUIRED`, `WRITE_POLICY_VIOLATION`, `PUBLISH_REPLICATION_FAILED`, `PUBLISH_PAYLOAD_TOO_LARGE`, `PUBLISH_INVALID_MESSAGE`
`SubscribeError`	`SUBSCRIBE_INVALID_TOPIC`, `SUBSCRIBE_ATTACH_FAILED`, `SUBSCRIBE_HANDLER_MISSING`
`KillError`	`KILL_INVALID_TOPIC`, `KILL_INVALID_MSGID`, `KILL_SIGN_FAILED`
`UnpubError`	`UNPUB_INVALID_TOPIC`, `UNPUB_PUBLIC_TOPIC`, `UNPUB_SIGN_FAILED`
`TouchError`	`TOUCH_INVALID_TOPIC`, `TOUCH_INVALID_MSGID`, `TOUCH_SIGN_FAILED`
`PullError`	`PULL_INVALID_MSGID`, `PULL_AXONS_UNREACHABLE`
`MetricsError`	`METRICS_AXONS_UNREACHABLE`
`UpgradeRequiredError`	`UPGRADE_REQUIRED` (bridge close code 4426)

The full taxonomy is in `ErrorCodes`:

```
import { ErrorCodes } from '@axona/protocol';
console.log(ErrorCodes.WRITE_POLICY_VIOLATION);  // 'WRITE_POLICY_VIOLATION'
```

Switch on `.code`, not `.message`:

```
try { await peer.pub(topic, msg, { signWith: me }); }
catch (err) {
  if (err.code === ErrorCodes.WRITE_POLICY_VIOLATION) { /* not the owner */ }
```

```
  else if (err.code === ErrorCodes.PUBLISH_NO_PUBLISH_IDENTITY) { /* name a signer */ }
  else throw err;
}
```

Wire-format helpers

Symbol	Notes
`isWireError(obj) → boolean`	does this look like an over-the-wire `AxonaError`?
`fromWire(obj) → AxonaError`	reconstruct the typed error (unknown classes fall back to `AxonaError`).
`err.toWire() → object`	`{ __axonaError: true, class, code, message, context }`.

Errors thrown inside a remote `onMessage` (reached via `peer.send`) survive serialization with their class and code intact.

---

9. Persistence

`PersistenceAdapter` (abstract class)

The storage contract. Implement it to plug in your own backend.

```
class PersistenceAdapter {
  async load(key)        { /* → previously-saved value or null */ }
  async save(key, value) { /* persist value */ }
  async delete(key)      { /* remove key */ }
}
```

Pass an instance as the `persist` option of the `AxonaPeer` constructor; the peer auto-checkpoints identity, subscriptions, synaptome, and hosting state.

`InMemoryPersistence`

Reference implementation (environment-neutral) used by tests and by the default `persist: false` path.

```
import { AxonaPeer, InMemoryPersistence } from '@axona/protocol';
const peer = new AxonaPeer({ nodeIdentity, domain, node, transport,
  persist: new InMemoryPersistence() });
```

`FilePersistence` / `IndexedDBPersistence` (sub-path imports)

Platform-specific implementations — sub-path imports only (so neither pulls `node:fs` into a browser bundle nor `globalThis.indexedDB` into Node):

```
import { FilePersistence }      from '@axona/protocol/persistence/file.js';      // Node
import { IndexedDBPersistence } from '@axona/protocol/persistence/indexeddb.js'; // browser

const peer = new AxonaPeer({ nodeIdentity, domain, node, transport,
  persist: new FilePersistence({ dir: './state' }) });
```

Transport + protocol surface

10. Contracts

Abstract base classes that transports / DHT implementations extend. Application code rarely uses these directly.

`Transport` (abstract class)

```
class MyTransport extends Transport {
  async start(localNodeId)         {}
  async stop()                     {}
  async send(peerId, type, body)   {}   // request/response
  async notify(peerId, type, body) {}   // fire-and-forget
  async openConnection(peerId)     {}
  async closeConnection(peerId)    {}
  isConnected(peerId)              {}
  getLatency(peerId)               {}   // RTT ms or -1
  onRequest(type, handler)         {}
  onNotification(type, handler)    {}
  onPeerDied(handler)              {}
}
```

Full surface in `@axona/protocol/contracts/Transport.js`.

`DHT` (abstract class)

The per-node routing/pub-sub contract `AxonaPeer` implements (`getSelfId`, `findKClosest`, `routeMessage`, `sendDirect`, `onRoutedMessage`, `lookup`, …).

`BootstrapService` (abstract class)

For peer discovery (signaling brokers, seed lists, DNS-SD). Rarely needed by app code.

---

11. Sim transport

In-process router for tests + simulators. Environment-neutral, ships in the main barrel.

`new SimNetwork({ latencyFn? })`

```
import { SimNetwork } from '@axona/protocol';
const net = new SimNetwork();                          // 0 ms edges
const net2 = new SimNetwork({ latencyFn: () => 50 });  // 50 ms each way
```

`simTransport({ network, identity, heartbeatMs?, ... }) → SimTransport`

Factory that builds + wires a `SimTransport` onto a `SimNetwork`.

```
import { SimNetwork, simTransport } from '@axona/protocol';
const network = new SimNetwork();
const t = simTransport({ network, identity: node, heartbeatMs: 0 });
await t.start(node.id);
await t.openConnection(other.id);
```

`heartbeatMs: 0` disables heartbeats (recommended for tests). `SimTransport` is also exported directly for `new SimTransport({...})`.

12. Web transport

The production browser transport (a WebSocket to the bridge for bootstrap + signaling, plus a WebRTC mesh). Sub-path import:

```
import { webTransport } from '@axona/protocol/transport/web/index.js';

const transport = webTransport({
  bridgeUrl:   'wss://bridge.axona.net',  // required
  identity:    node,                      // from createNodeIdentity - signs the handshake
  peerVersion: '3.5.0',                   // your app version (gated by the bridge)
  reconnect:   true,
});
await transport.start();                  // resolves after the bridge handshake
```

Beyond the `Transport` contract it exposes an observability surface: `transport.bridgeState`, `bridgeInfo`, `bridgeRtt`, `bridgeNodeId`, `onBridgeState(cb)`, `onWelcome(cb)`, `boundPeers()`, `reconnectNow()`.

Each mesh link's `axona/5` proof is bound to its DTLS certificate fingerprint, so a fingerprint-rewriting bridge cannot transparently MITM "direct" peer traffic — the bridge is trusted for signaling only.

---

13. Handshake

Wire-version handshake helpers used by transports on a fresh signaling channel. Most app code never touches these — they're plumbed into the transport factories.

```
buildClientHello({ version, wireVersion?, capabilities? })          // → client → server frame
buildServerHello({ version, wireVersion?, minPeerVersion, downloadUrl }) // → server → client frame
parseHello(frame)                                                   // → parsed hello
parseVersion(str)                                                   // → { major, minor, patch }
compareVersions(a, b)                                               // → -1 | 0 | 1
wireCompatible(a, b)                                                // → boolean
performClientHandshake(channel, opts)                               // → handshake result
performServerHandshake(channel, opts)
```

The constants `WIRE_VERSION`, `KERNEL_VERSION`, `UPGRADE_CLOSE_CODE` come from the same module — see §23.

---

14. Authenticated-identity handshake (axona/5)

Re-exported so consumers driving their own channel lifecycle (the bridge's embedded peer, custom transports) can build/verify authenticated hellos with the same primitive the web transport uses.

```
buildAuthHello(opts)                  // → signed hello proving nodeId ownership
verifyAuthHello(hello, opts)          // → verification result
pubkeyMatchesNodeId(pubkey, nodeId)   // → boolean (BIND check)
makeNonce()                           // → fresh challenge nonce
cbvFromNonces(...)                    // → channel-binding value from nonces
cbvFromFingerprints(...)              // → channel-binding value from DTLS fingerprints
AUTH_PROTO                            // 'axona/5'
```

The proof binds (BIND: pubkey hashes to the id), (POSSESS: Ed25519 signature), and (CHANNEL: bound to this live connection) so a captured proof can't be replayed onto another link.

15. Bridge directory

Bridges advertise on a public topic; clients collect them at launch and fail over.

```
BRIDGE_DIRECTORY_TOPIC              // 'axona:bridge-directory'
BRIDGE_ENTRY_MAX_AGE_MS             // 48 h
buildBridgeEntry({ url, lat, lng, label?, ver?, turn?, ts? })  // → directory entry
validateBridgeEntry(msg)            // → boolean (well-formed + fresh)
rankBridges({ ... })                // → ordered candidate list (reputation + proximity)
haversineKm(a, b)                   // → great-circle km between two { lat, lng }
```

```
import { BRIDGE_DIRECTORY_TOPIC, buildBridgeEntry } from '@axona/protocol';
const entry = buildBridgeEntry({ url: 'wss://bridge.axona.net', lat: 38, lng: -77 });
await peer.pub({ name: BRIDGE_DIRECTORY_TOPIC, region: 'useast' }, entry, { signWith: me });
```

16. Low-level pub/sub builders

The signed wire records behind `peer.kill` / `touch` / `unpub`. Apps use the `peer.*` methods; these are for protocol implementors and custom roots.

```
buildKill({ topicId, msgId, ts?, seq?, identity })                    // → signed kill
verifyKill(kill)                                                       // → verification result
KILL_DOMAIN     // 'axona:pubsub-kill:v1'

buildTouch({ topicId, msgId, ts?, seq?, identity })                   // → signed touch
verifyTouch(touch)
TOUCH_DOMAIN    // 'axona:pubsub-touch:v1'

buildUnpub({ topicId, topicName, ownerNodeId, destroy?, ts?, seq?, identity }) // → signed unpub
verifyUnpub(unpub)
UNPUB_DOMAIN    // 'axona:pubsub-unpub:v1'
```

`AxonaManager` (the pub/sub state machine), `AxonaDomain`, `NeuronNode`, `Synapse`, `Subscription`, `DHTNode`, and `GEO_CELL_BITS` (= 8) are also exported for implementors building custom engines.

Low-level utilities

17. Ed25519 helpers

Web Crypto Ed25519 wrappers — useful for signing/verifying outside `buildEnvelope`.

```
const { publicKey, privateKey } = await generateKeyPair({ extractable: true });
const pubHex  = await exportPublicKey(publicKey);   // 64-char hex
const pubKey  = await importPublicKey(pubBytesOrHex);
const sig     = await sign(privateKey, dataBytes);  // Uint8Array(64)
const ok      = await verify(pubKeyOrBytes, dataBytes, sig);

const signer   = makeSigner(privateKey);   // (bytes) => Promise<sig>
const verifier = makeVerifier(pubKey);      // (bytes, sig) => Promise<boolean>
```

Implementation-agnostic: substitute `@noble/ed25519` on runtimes without Web Crypto Ed25519 (Chrome <110, Safari <17, Firefox <130, Node <20).

18. Hex / ID math

264-bit identifier math — nodeId and topicId share the same keyspace (`[8-bit S2 prefix] || [256-bit hash]`).

```
ID_BITS                    // 264
HASH_BITS                  // 256
S2_BITS                    // 8
HEX_CHARS                  // 66
MAX_ID                     // BigInt (2**264 - 1)
MAX_HASH                   // BigInt (2**256 - 1)
MAX_S2                     // 255

toHex(bigint)              // → 66-char lowercase hex
fromHex(hex)               // → BigInt
isHexId(s)                 // → boolean (valid 66-char hex id)
assembleId(s2Prefix, hash) // → BigInt - 8-bit prefix || 256-bit hash
extractS2Prefix(idBig)     // → number 0..255 (top byte)
extractHash(idBig)         // → BigInt (bottom 256 bits)
s2PrefixOfHex(hex)         // → number 0..255
xorDistance(a, b)          // → BigInt
stratumOf(a, b)            // → 0..264 - leading-zero count of (a ^ b)
clz264(bigint)             // → 0..264 - leading-zero count
randomU256()               // → BigInt - cryptographically random
```

19. S2 geographic cells

The 8-bit S2 cell that occupies the top byte of every id. The cellId matches the top 8 bits of Google S2's level-3 cell ID (full interop).

```
geoCellId(lat, lng, bits)  // → integer - S2 cell ID at the given bit depth
geoCellCenter(cellId)      // → { lat, lng } - cell center
geoCellCorners(cellId)     // → corner coordinates
geoCellFace(cellId)        // → 0..5 - which cube face
geoCellSubCenters(cellId)  // → [center0, center1] - the two level-3 sub-cell centers
geoCellHalf(lat, lng)      // → 0 | 1 - which sub-cell a point falls in
isValidCellId(cellId)      // → boolean (in [0, 192))
S2_FACES                   // 6
S2_CELL_COUNT              // 192
S2_RESERVED_FROM           // 192
```

20. Region names

Each of the 192 region codes carries exactly one human-readable name, so a region always presents the same label. Where a cell straddles ocean and land the land name wins; a multi-country cell takes its dominant city.

```
REGION_NAMES               // frozen string[] indexed by code [0,192)
regionName(code)           // → string | null     e.g. regionName(0x89) → 'useast'
regionNames(code)          // → [name] | null      deprecated one-element shim
regionCode(name)           // → number | null      inverse (canonical code for a multi-cell name)
```

```
resolveRegion(nameOrCode)      // → number | null      accepts 'useast' | '0x89' | '137' | 137
regionNameForLatLng(lat, lng) // → string             region name for a coordinate
regionCenter(nameOrCode)       // → { lat, lng } | null cell center (default placement for topics)
POPULATED_REGIONS              // frozen [{ code, name }] for every non-open-ocean cell

import { regionCode, regionCenter, POPULATED_REGIONS } from '@axona/protocol';
regionCode('useast');          // 137
regionCenter('useast');        // { lat, lng } - mint a node identity in a region
POPULATED_REGIONS.length;      // count of real, inhabited cells (for a region picker)
```

Names match `/^[a-z0-9_]{1,8}$/`; open-ocean cells are `<ocean3>_<hex>` (`pac_68`, `atl_0a`, …) and are excluded from `POPULATED_REGIONS`.

---

21. Geo helpers

```
haversine(lat1, lng1, lat2, lng2)        // → km between two points
roundTripLatency(lat1, lng1, lat2, lng2) // → ms - rough fiber estimate
randomU32()                               // → 0..2**32 - 1
```

The remaining `geo.js` helpers (continent detection, XOR routing-table builders, etc.) are reachable via `@axona/protocol/utils/geo.js`.

---

22. Proof-of-work scaffolding

E-1 / Stage 2 transport + publish PoW. **Inert at difficulty 0** — every nonce is `''` and verification passes trivially until difficulty is raised. Apps don't normally touch these; identities carry their PoW nonce automatically.

```
POW_DOMAIN                                  // 'axona:pow:v1'
POW_DIFFICULTY                              // { transport: 0, publish: 0 } (frozen defaults)
powDifficulty(role)                         // → current difficulty for 'transport' | 'publish'
setPowDifficulty(role, n)                   // set difficulty (testing / calibration)
resetPowDifficulty()                        // restore defaults
powMint({ pubkeyHex, role?, difficulty?, maxTries? })  // → Promise<nonce string>
powVerify({ pubkeyHex, nonce, role?, difficulty? })    // → Promise<boolean>
powBits({ pubkeyHex, nonce, role? })        // → Promise<number> leading-zero bits of the puzzle hash
powCalibrate({ ms? })                       // → Promise<calibration> bits/ms estimate
```

The puzzle hashes `H(domain || role || pubkey || nonce)` — difficulty is on a **separate puzzle hash, never on the node address**, so raising it introduces no keyspace skew.

---

23. Constants

```
WIRE_VERSION         // '3.0'      - wire format major.minor (bridges gate on this)
KERNEL_VERSION       // '3.5.0'    - kernel semver (npm release tag)
AUTH_PROTO           // 'axona/5'  - authenticated-identity handshake tag
UPGRADE_CLOSE_CODE   // 4426       - WebSocket close code for a version mismatch
ENVELOPE_DOMAIN      // 'axona:pubsub-envelope:v2'
MAX_PUBLISH_SKEW_MS  // 300000     - freshness window (5 min)
ID_BITS              // 264
HEX_CHARS            // 66
```

`WIRE_VERSION` is what bridges enforce gates against; `KERNEL_VERSION` is informational.

`ANONYMOUS` (sentinel)

The sentinel for an intentionally unsigned publish. Anonymity must be explicit — omitting a signer is an error, never silent anonymity.

```
import { ANONYMOUS } from '@axona/protocol';
await peer.pub({ region: 'useast', name: 'lobby' }, msg, { signWith: ANONYMOUS });
```

A word on stability

`@axona/protocol` v3.x is stable for the application surface (§§1–9). The transport + protocol surface (§§10–16) is stable but expect occasional additions. Low-level utilities (§§17–23) may grow but won't change incompatibly.

For changelog and migration notes, see https://github.com/axona-net/axona-protocol/releases.